Annotation Guidelines: Open-Ended Survey Responses

Scope

These guidelines describe how we code the three open-ended survey questions:

- **Q33:** Describe a situation where AI tools helped reduce technical debt and improved software quality in your development work.
- **Q34:** Describe a situation where AI-generated code or AI-assisted development introduced technical debt or caused problems in your software project.
- **Q35:** What improvements would you like to see in AI coding assistants to reduce the risk of AI-generated technical debt?

Respondents are identified as S1–S137, following the row order in data/survey_responses.csv.

Unit of Analysis

The unit of analysis is one respondent's complete answer to one question. A single answer can receive more than one theme.

Coding Procedure

1. **Familiarization.** Two annotators read all responses to a question before coding.
2. **Initial coding.** The annotators derive themes inductively from the responses rather than from a predefined list.
3. **Codebook development.** The annotators consolidate similar codes into themes and write a one-sentence definition for each theme (see the codebooks below).
4. **Coding.** The annotators assign every applicable theme to each response, using the definitions in the codebook.
5. **Comparison and resolution.** The annotators compare their coding and resolve disagreements through discussion. When a disagreement reveals an unclear definition, they revise the definition and re-check affected responses.

Coding Rules

- **Assign all applicable themes.** If a response describes several distinct issues or benefits, assign one theme for each.

- **Code only what the respondent states.** Do not infer causes, consequences, or intentions that the response does not express.
- **Miscellaneous.** Use *Miscellaneous* for substantive but generic responses that no other theme covers.
- **Uncoded responses.** Leave blank, irrelevant, or non-substantive responses (e.g., "N/A", "nothing") uncoded.

Reporting

For each theme, we report the number of responses (n) that received the theme and the percentage of **coded** responses: 118 for Q33, 109 for Q34, and 112 for Q35. Because responses can receive multiple themes, percentages do not sum to 100%.

The coded data appear in analysis/survey_coded_responses.csv.

Codebook: Q33 (AI reduced technical debt; 118 coded responses)

Theme	Definition	n
Refactoring	Restructuring existing code without changing its behavior.	46
Productivity	Increasing development output while reducing time and effort.	37
Code Quality & Maintainability	Improving code quality attributes, including readability, consistency, reliability, and ease of maintenance.	36
Test Generation	Generating test cases/test scripts to verify software behavior.	31
Code Generation	Generating source code, boilerplate, or scripts from specifications or prompts.	31
Code Comprehension	Understanding the structure, behavior, and logic of the code.	25
Code Review	Examining source code to identify defects and recommend improvements.	18
Architecture Design	Defining system structure, components, interfaces, and their interactions.	18
Debugging	Identifying, diagnosing, and correcting software defects.	18
Documentation	Creating and updating documentation and code comments.	17
Legacy System Maintenance	Modifying and maintaining existing legacy software systems.	15

Theme	Definition	n
Planning	Defining requirements, decomposing tasks, and organizing development activities.	14
Prototyping	Developing preliminary implementations to evaluate ideas/feasibility.	6
Performance Optimization	Improving execution speed, resource utilization, and computational efficiency.	6
Code Migration	Adapting software to different languages, frameworks, or platform versions.	5
Miscellaneous	Generic responses not covered by other themes.	8

Codebook: Q34 (AI introduced technical debt; 109 coded responses)

Theme	Definition	n
Rework	Additional effort to rewrite, refactor, correct, or revert AI-generated code.	33
Maintainability	Code that is difficult to understand, modify, or maintain.	26
Context Misalignment	Generated code that overlooks project requirements, constraints, or context.	23
Architectural Inconsistency	Violations of established architecture, design conventions, or coding standards.	22
Overengineering	Introduction of unnecessary complexity, abstractions, components, or functionality.	21
Incorrect or Hallucinated Output	Incorrect, fabricated, or irrelevant AI-generated code or information.	20
Software Reliability	New bugs, regressions, or runtime failures caused by AI-generated code.	18

Theme	Definition	n
AI Overreliance	Excessive AI reliance leading to cognitive dependence and weakened developer knowledge and skills.	16
Code Duplication	Repeated code or logic across functions, files, or modules.	15
Review Overhead	Extra effort to review and validate AI-generated code.	11
Performance	Inefficient code causing slow execution, excessive resource use, or performance bottlenecks.	9
Testing	Missing, inadequate, or unreliable tests, or AI actions that disrupt test suites.	8
Debugging Difficulties	Challenges in debugging AI-generated code due to limited context and added complexity.	7
AI Tool Limitations	Constraints and shortcomings of specific AI tools, models, and versions.	5
Domain-Specific Knowledge Gaps	Limited understanding of specialized domains, technologies, and their requirements.	5
Security	Vulnerable code or suggestions of unsafe practices.	4
Dependency Management	Version incompatibilities, dependency conflicts, or unnecessary dependencies.	4
Team Coordination	Inconsistent AI-use practices among the team members.	4
Infrastructure	Incorrect, broken, or oversized infrastructure configurations.	2
Legacy System Migration	Incorrect transformations during legacy migrations or upgrades.	2
Miscellaneous	Generic responses not covered by other themes.	14

Codebook: Q35 (Desired improvements; 112 coded responses)

Theme	Definition	n
Architectural Awareness	Adhering to existing system architecture, design patterns, and coding conventions.	30
Risk Identification	Identifying and warning about uncertain, risky, or potentially problematic existing/generated code.	18
Coding Best Practices	Following established coding standards and principles for readable, modular, and maintainable code.	17
Testing Support	Generating tests and identifying edge cases to verify code correctness.	17
Requirements Understanding	Correctly interpreting requirements, business rules, and developer intent.	16
Output Reliability	Ensuring accurate, consistent, and contextually relevant outputs while minimizing hallucinations.	13
Explainability	Explaining generated code, design decisions, and trade-offs.	13
Clarification Requests	Seeking clarification when requirements or instructions are ambiguous or incomplete.	13
Code Reuse	Reusing existing components and preventing code duplication.	13
Automated Code Validation	Automatically checking generated code for correctness and quality before acceptance.	12
Developer Skill Development	Enhancing developer skills through learning, training, and effective use of AI tools.	11
Security Validation	Identifying security vulnerabilities in the generated code.	11

Theme	Definition	n
Human Oversight	Maintaining developer control through review, validation, and guidance of AI-generated code.	9
Structured Planning	Outlining implementation steps and specifications before code generation.	9
Debt Detection	Identifying technical debt, unnecessary complexity, and maintainability concerns.	8
Context Management	Maintaining, retrieving, and effectively utilizing relevant project context to ensure continuity and consistency across tasks and interactions.	8
Change Scope Control	Limiting modifications to the requested scope while preserving existing functionality.	6
Documentation	Generating and maintaining documentation for code changes.	5
Cost & Accessibility	Minimizing token-related constraints and computational costs while improving the affordability and accessibility of AI tools.	3
Miscellaneous	Generic responses not covered by other themes.	3